Lab Sheet — TP 2

Recursion, Memoisation and Iteration

Corresponds to: Lecture 3 (see TD 2)

Session Objectives

- Observe the exponential blow-up of naive recursion experimentally.
- Apply memoisation with functools.lru_cache and measure the speed-up.
- Implement an iterative bottom-up version and compare its scalability.

Exercise 1 — Guided (done together in session)

Complete the memoised version of the Fibonacci function below, then compare its number of calls and running time against the naive version for n = 30.

```
import functools, time

call_count = 0
def fib_naive(n):
    global call_count
    call_count += 1
    if n <= 1:
        return n
    return fib_naive(n - 1) + fib_naive(n - 2)

@functools.lru_cache(maxsize=None)
def fib_memo(n):
    # TODO: same recursive definition as fib_naive, but decorated with lru_cache
    pass

call_count = 0
start = time.perf_counter()
print(fib_naive(30))
print('naive calls:', call_count, 'time:', time.perf_counter() - start)

start = time.perf_counter()
print(fib_memo(30))
print('memo time:', time.perf_counter() - start)
```

Exercise 2 — Autonomous (in session, self-checked with the tests provided)

Implement an iterative, bottom-up version of Fibonacci running in Θ(n) time and O(1) auxiliary space, then test it for correctness against fib_memo, and observe what happens when computing a very large index with each version.

```
def fib_iterative(n):
    # TODO: compute fib(n) iteratively in O(1) space
    pass

# Correctness tests
for n in range(20):
    assert fib_iterative(n) == fib_memo(n)
print('All correctness tests passed')

# Try a large index
print(fib_iterative(100000))       # should work fine
# print(fib_memo(100000))          # try uncommenting this: what happens, and why?
```

Exercise 3 — Optional, to take further at home

The Tribonacci sequence is defined by T(0) = 0, T(1) = 0, T(2) = 1, and T(n) = T(n-1) + T(n-2) + T(n-3) for n ≥ 3. Write both a memoised recursive version and an iterative Θ(n) version, check they agree for n = 0..30, and time both for n = 100 000.